SIMATS SCHOOL OF ENGINEERING

Approved By AICTE | IET-UK Accreditation

Course Code: DSA03	Course Name: Natural Language Processing
Branch: CSE	Year : 2025

Name: R.Rohith Manikanta
Reg no: 192425326

Assignment 1

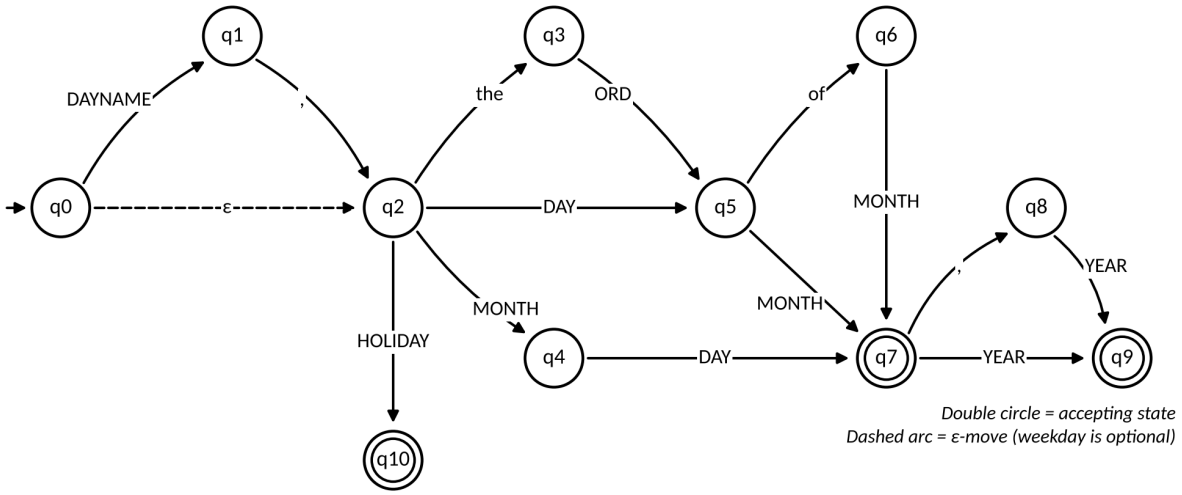
1. Design an FSA that recognizes simple date expressions like March 15, the 22nd of November, Christmas. You should try to include all such "absolute" dates, (e.g. not "deictic" ones relative to the current day like the day before yesterday). Each edge of the graph should have a word or a set of words on it. You should use some sort of shorthand for classes of words to avoid drawing too many arcs (e.g., furniture→ desk, chair, table). (10M) (CO1) (BLT4)
2. Write regular expressions for the following languages: You may use either Perl notation or the minimal "algebraic" notation of Sec. 2.3, but make sure to say which one you are using. By "word", we mean an alphabetic string separated from other words by white space, any relevant punctuation, line breaks, and so forth. (10M) (CO1) (BLT4)
   a. the set of all alphabetic strings.
   b. the set of all lowercase alphabetic strings ending in a b.
   c. the set of all strings with two consecutive repeated words (e.g., "Humbert Humbert" and "the the" but not "the bug" or "the big bug").
   d. the set of all strings from the alphabet a,b such that each a is immediately preceded and immediately followed by a b.
   e. all strings that start at the beginning of the line with an integer and that end at the end of the line with a word.
   f. all strings which have both the word grotto and the word raven in them. (but not, for example, words like grottos that merely contain the word grotto).
   g. write a pattern which places the first word of an English sentence in a register.

   Deal with punctuation.
3. Computing minimum edit distances by hand, figure out whether drive is closer to brief or to

divers, and what the edit distance is. You may use any version of distance that you like. (10M) (CO1) (BLT4)

Answer 1. FSA for absolute date expressions

The automaton below accepts absolute dates such as *March 15*, *the 22nd of November*, *Monday*, *March 15, 2025* and *Christmas*. Every edge carries a word or a class of words; the classes are defined under the figure. q0 is the start state and the double-circled states q7, q9 and q10 are accepting.



MONTH	January | February | … | December, and abbreviations Jan | Feb | Mar | Apr | Jun | Jul | Aug | Sep | Sept | Oct | Nov | Dec (optionally followed by ".")
DAYNAME	Monday | Tuesday | Wednesday | Thursday | Friday | Saturday | Sunday
ORD	1st | 2nd | 3rd | 4th | … | 31st, or first | second | third | … | thirty-first
DAY	ORD or a plain number 1 | 2 | … | 31 (so "15" and "15th" are both accepted)
YEAR	any four-digit number such as 1999, 2025
HOLIDAY	Christmas | Christmas Day | Christmas Eve | New Year's Day | New Year's Eve | Valentine's Day | Halloween | Easter | Thanksgiving | Independence Day | Diwali | Pongal

Sample traces: "March 15" = q0 ε q2 MONTH q4 DAY q7; "the 22nd of November" = q0 ε q2 the q3 ORD q5 of q6 MONTH q7; "Monday, March 15, 2025" = q0 DAYNAME q1 "," q2 MONTH q4 DAY q7 "," q8 YEAR q9; "Christmas" = q0 ε q2 HOLIDAY q10. The ε-arc (q0 to q2) makes the weekday optional, and the shorthand classes keep the figure to 15 arcs instead of hundreds.

Answer 2. Regular expressions (Perl notation is used throughout)

Here "word" means a maximal alphabetic string, so **\b** (word boundary) is used wherever a pattern has to find words inside running text, and punctuation is treated as a separator.

a. All alphabetic strings:

/^[A-Za-z]+$/

(to pick words out of text: /\b[A-Za-z]+\b/)

b. Lowercase alphabetic strings ending in b:

/^[a-z]*b$/

(in text: /\b[a-z]*b\b/; the single string “b” is accepted)

c. Two consecutive repeated words:

/\b([A-Za-z]+)\s+\1\b/i

The first \b stops a match from starting inside a word, the back-reference \1 repeats the captured word, and the final \b rejects “the theory”. \s+ also covers a line break between the two words. It matches “the the” and “Humbert Humbert” but not “the bug” or “the big bug”.

d. Strings over {a,b} where every a has a b immediately before and after it:

/^(b(ab|b)*)?$/

The string must start with b, and after that it is built only from the units “b” and “ab”, so each a is preceded by the b that ends the previous unit and followed by the b inside its own unit. It accepts b, bb, bab, babbab and (vacuously) the empty string, and rejects a, ab, ba, baab.

e. Lines that start with an integer and end with a word:

/^[0-9]+\b.*\b[A-Za-z]+$/

(tolerating final punctuation: /^[0-9]+\b.*\b[A-Za-z]+[.,;:!?]*$/)

^ and $ anchor to the start and end of the line, e.g. “25 students enrolled” matches while “25” alone does not.

f. Strings containing both the word grotto and the word raven (in either order):

/\bgrotto\b.*\braven\b|\braven\b.*\bgrotto\b/

The alternation covers both orders, and \b on each side rejects “grottos” and “ravens”. An equivalent form using look-aheads is /^(?=.*\bgrotto\b)(?=.*\braven\b)/.

g. Place the first word of each English sentence in a register:

/(?:^|[.!?]["')]*\s+)["'(]*([A-Za-z]+(?:'[A-Za-z]+)?)/g

A sentence starts at the beginning of the text or after “.”, “!” or “?” (plus any closing quote or bracket) and whitespace. Opening quotes or brackets are skipped, and the parentheses around the word capture it into register \1 (Perl: $1). Punctuation is therefore never captured, and contractions such as “Don’t” are kept whole. Abbreviations like “Dr.” can fool it.

Answer 3. Minimum edit distance: drive vs. brief and drive vs. divers

Version used: Levenshtein distance with insertion cost 1, deletion cost 1 and substitution cost 2 (a substitution counts as a deletion plus an insertion). Each cell D[i,j] = min(D[i-1,j]+1, D[i,j-1]+1, D[i-1,j-1] + (0 if the letters match, else 2)), with D[i,0]=i and D[0,j]=j. Rows are the source **drive** and columns the target.

(i) drive → brief : distance = 4

	#	b	r	i	e	f
#	0	1	2	3	4	5
d	1	2	3	4	5	6
r	2	3	2	3	4	5
i	3	4	3	2	3	4
v	4	5	4	3	4	5
e	5	6	5	4	3	**4**

(ii) drive → divers : distance = 3

	#	d	i	v	e	r	s
#	0	1	2	3	4	5	6
d	1	0	1	2	3	4	5
r	2	1	2	3	4	3	4
i	3	2	1	2	3	4	5
v	4	3	2	1	2	3	4
e	5	4	3	2	1	2	**3**

Alignments. drive → brief: substitute d→b (2), r, i match, delete v (1), e matches, insert f (1) = **4**. drive → divers: d matches, delete r (1), i, v, e match, insert r (1), insert s (1) = **3**.

Conclusion: drive is closer to divers, with a minimum edit distance of **3**, whereas the distance to brief is 4. (With plain unit-cost Levenshtein both would tie at 3, so the substitution-cost-2 version is the one that separates them.)